

Trabajo Práctico 2 - Bases de Datos

Alumno: Fabrizio Aguzzi

27 de marzo de 2025

- **Objetivos:** El objetivo del siguiente TP es poder crear y gestionar las sentencias de SQL Media que corresponden al ANSI SQL.
- **Consigna:** El alumno deberá subir un archivo en formato Word o PDF con las sentencias de cada uno de los 54 puntos que se detallan a continuación teniendo en cuenta el siguiente DER de la unidad 1, en caso de tener alguna duda o no poder hacer algún ejercicio comentarlo para revisión del docente.

1. Escribir las sentencias de CREATE TABLE, PRIMARY KEY, y FOREIGN KEY, para construir la base de datos.

- Primero creé la tabla Categoria:

```
CREATE TABLE Categoria (  
    Tipo VARCHAR2(3) PRIMARY KEY,  
    Descripcion VARCHAR2(255)  
);
```

- Luego creé la tabla Bodega:

```
CREATE TABLE Bodega(  
    Tipo_B VARCHAR(2) PRIMARY KEY,  
    Descripcion VARCHAR2(255)  
);
```

- Posteriormente creé la tabla Vino:

```
CREATE TABLE Vino (  
    Codigo NUMBER PRIMARY KEY,  
    Descripcion VARCHAR2(255),  
    Anio_Elaboracion NUMBER(4),  
    Categoria VARCHAR2(3),  
    Bodega VARCHAR2(2),  
    CONSTRAINT fk_Categoria FOREIGN KEY (Categoria) REFERENCES  
        Categoria(Tipo),  
    CONSTRAINT fk_Bodega FOREIGN KEY (Bodega) REFERENCES  
        Bodega(Tipo_B)  
);
```

- Creación de la Tabla Clientes:

```
CREATE TABLE Cliente (  
    ID_CLIENTE NUMBER PRIMARY KEY,  
    Nombre_y_Apellido VARCHAR2(50),  
    Razon_Social VARCHAR2(50),  
    e_mail VARCHAR2(50),  
    Tarjeta_de_credito VARCHAR2(20)  
);
```

- Creación de la Tabla Pedidos:

```
CREATE TABLE Pedido (
  Codigo VARCHAR2(2),
  ID_CLIENTE NUMBER(22),
  fecha_pedido DATE,
  cantidad INT,
  Precio_Total DECIMAL(10,2),
  CONSTRAINT fk_Vino FOREIGN KEY (Codigo) REFERENCES Vino(
    Codigo),
  CONSTRAINT fk_Cliente FOREIGN KEY (ID_CLIENTE) REFERENCES
    Cliente(ID_CLIENTE),
  PRIMARY KEY (Codigo, ID_CLIENTE, fecha_pedido)
);
```

- Creación de la Tabla Categoría del Proveedor:

```
CREATE TABLE Categoria_proveedor (
  Codigo_prov VARCHAR2(2) PRIMARY KEY,
  Descripcion VARCHAR(250),
  Cant_de_vino_Vendido INT,
  Facturacion_anual INT,
  Ventas_anuales INT
);
```

- Creación de la Tabla Proveedor:

```
CREATE TABLE proveedor (
  Codigo_proveedor VARCHAR2(2) PRIMARY KEY,
  Descripcion VARCHAR(250),
  Codigo_prov VARCHAR2(2),
  CONSTRAINT fk_Categoria_proveedor FOREIGN KEY (Codigo_prov)
    REFERENCES Categoria_proveedor(Codigo_prov)
);
```

- Creación de la Tabla Compras:

```
CREATE TABLE compra (
  Codigo_proveedor VARCHAR2(2),
  Codigo VARCHAR2(2),
  Fecha_de_Compra DATE,
  Cantidad INT,
  CONSTRAINT fk_proveedor FOREIGN KEY (Codigo_proveedor)
    REFERENCES proveedor(Codigo_proveedor),
  CONSTRAINT fk_vino_compra FOREIGN KEY (Codigo) REFERENCES
    vino(Codigo)
);
```

2. Utilizar las sentencias de INSERT para que las tablas no estén vacías, con los ejemplos que están en el foro.

```
INSERT INTO CATEGORIA VALUES ('TIN','TINTOS');
INSERT INTO CATEGORIA VALUES ('BLA','BLANCOS');
```

```
INSERT INTO CATEGORIA VALUES ('ROS','ROSADOS');
INSERT INTO CATEGORIA VALUES ('ESP','ESPUMANTES');
```

```
INSERT INTO BODEGA VALUES ('AL','ALAMOS');
INSERT INTO BODEGA VALUES ('AF','ACHAVAL FERRER');
INSERT INTO BODEGA VALUES ('BA','BODEGA AMALAYA');
INSERT INTO BODEGA VALUES ('BR','BODEGA ARGENTO');
```

```
INSERT INTO VINO VALUES ('LM','LINDAFLORES MALBEC', 2000, 'TIN', 'AL');
INSERT INTO VINO VALUES ('MM','MARTIN BRUNO MALBEC', 1998, 'TIN',
'AF');
INSERT INTO VINO VALUES ('VB','VIA BLANCA MALBEC', 1996, 'TIN',
'BA');
INSERT INTO VINO VALUES ('GL','GRETA LA SALVAJE SEMILLON', 1998,
'BLA', 'BA');
INSERT INTO VINO VALUES ('TR','TRUMPETER RESERVE DOUX', 1996,
'BLA', 'BA');
INSERT INTO VINO VALUES ('AV','ALTA VISTA SAUVIGNON', 1995, 'BLA',
'AF');
INSERT INTO VINO VALUES ('AS','AMANSADO CLASSIC ROSE', 2002, 'BLA',
'AF');
INSERT INTO VINO VALUES ('ZT','ZORZAL TERROIR', 2004, 'BLA', 'AF');
INSERT INTO VINO VALUES ('PF','PINK FLOOD DE JUANCHI', 2006, 'BLA',
'AF');
```

```
INSERT INTO CLIENTE VALUES (1, 'MARINAO GECIK', '15-4-567-8976', 'MGE-
CIK@GMAIL.COM', '3456-5678-3245-7654');
INSERT INTO CLIENTE VALUES (2, 'RICARDO PEREZ', '15-4-356-8900', 'RPE-
REZ@GMAIL.COM', '1234-5678-6789-2345');
INSERT INTO CLIENTE VALUES (3, 'SUSANA VAINSHTEIN', '15-4-765-1234',
'SVAINSHTEIN@GMAIL.COM', '9876-5092-3456-1342');
INSERT INTO CLIENTE VALUES (4, 'PABLO PATRIZI', '15-4-894-1334', 'PPA-
TRIZI@GMAIL.COM', '9986-5077-3896-1652');
```

```
INSERT INTO PEDIDO VALUES ('LM', '1', TO_DATE('21/06/2018','DD/MM/YYYY'),
100, 150);
INSERT INTO PEDIDO VALUES ('MM', '1', TO_DATE('21/06/2018','DD/MM/YYYY'),
100, 200);
```

```
INSERT INTO PEDIDO VALUES( 'AV', '2', TO_DATE('21/05/2018','DD/MM/YYYY'),  
130, 250);
```

```
INSERT INTO PEDIDO VALUES( 'AS', '2', TO_DATE('21/05/2018','DD/MM/YYYY'),  
120, 300);
```

```
INSERT INTO PEDIDO VALUES( 'GL', '3', TO_DATE('21/03/2019','DD/MM/YYYY'),  
140, 100);
```

```
INSERT INTO PEDIDO VALUES( 'TR', '3', TO_DATE('21/03/2019','DD/MM/YYYY'),  
120, 120);
```

```
INSERT INTO CATEGORIA_PROVEEDOR VALUES ( 'AC', 'ALTA CALIDAD',  
100, 1000000, 30000);
```

```
INSERT INTO CATEGORIA_PROVEEDOR VALUES ( 'BA', 'BAJA CALIDAD',  
200,0,0);
```

```
INSERT INTO PROVEEDOR VALUES ( 'LR', 'LA RURAL', 'AC');
```

```
INSERT INTO PROVEEDOR VALUES ( 'CO', 'COYANCO', 'AC');
```

```
INSERT INTO PROVEEDOR VALUES ( 'MO', 'MONETVIEJO', 'BA');
```

```
INSERT INTO PROVEEDOR VALUES ( 'RO', 'ROLLAND', 'BA');
```

```
INSERT INTO COMPRA VALUES( 'LR', 'LM', TO_DATE('21/06/2017','DD/MM/YYYY'),  
2000);
```

```
INSERT INTO COMPRA VALUES( 'LR', 'MM', TO_DATE('21/06/2017','DD/MM/YYYY'),  
2000);
```

```
INSERT INTO COMPRA VALUES( 'MO', 'GL', TO_DATE('21/09/2017','DD/MM/YYYY'),  
1500);
```

```
INSERT INTO COMPRA VALUES( 'MO', 'TR', TO_DATE('21/09/2017','DD/MM/YYYY'),  
1500);
```

```
INSERT INTO COMPRA VALUES( 'RO', 'GL', TO_DATE('21/05/2017','DD/MM/YYYY'),  
1000);
```

```
INSERT INTO COMPRA VALUES( 'RO', 'TR', TO_DATE('21/05/2017','DD/MM/YYYY'),  
800);
```

-
3. Ejecutar una sentencia que muestre todos los campos de una tabla.

```
SELECT * FROM Pedido;
```

129 `SELECT * FROM Pedido;`

CODIGO	ID_CLIENTE	FECHA_PEDIDO	CANTIDAD	PRECIO_TOTAL
MM	1	6/21/2018	100	200
AS	2	5/21/2018	120	300
AV	2	5/21/2018	130	250
LM	1	6/21/2018	100	150
TR	3	3/21/2019	120	120
GL	3	3/21/2019	140	100

5 rows returned in 0.02 seconds [Download](#)

4. Ejecutar una sentencia que muestre 2 campos de otra tabla.

```
SELECT ID_CLIENTE,E_MAIL FROM CLIENTE;
```

133 `SELECT ID_CLIENTE,E_MAIL FROM CLIENTE;`

ID_CLIENTE	E_MAIL
1	MGECK@GMAIL.COM
2	RPerez@GMAIL.COM
4	PPATRIZ@GMAIL.COM
3	SVAINSHTEIN@GMAIL.COM

4 rows returned in 0.01 seconds [Download](#)

5. Ejecutar una sentencia que pueda multiplicar 2 campos de una tabla del modelo.

```
SELECT CANTIDAD * PRECIO_TOTAL AS TOTAL_VENTAS FROM PEDIDO;
```

3 `SELECT CANTIDAD * PRECIO_TOTAL AS TOTAL_VENTAS FROM PEDIDO;`
4

TOTAL_VENTAS
20000
36000
32500
15000
14400
14000

6 rows returned in 0.02 seconds [Download](#)

6. Siguiendo con el punto 5 que calcule, además, un 35 % del precio unitario.

```
SELECT CANTIDAD * PRECIO_TOTAL AS TOTAL_VENTAS FROM PEDIDO;
```

```

5
6 SELECT (PRECIO_TOTAL / NULLIF(CANTIDAD, 0)) * 0.35 AS Precio_Unitario_35 FROM PEDIDO;
7

```

PRECIO_UNITARIO_35
.7
.875
.673076923076923076923076923076922
.525
.35
.25

6 rows returned in 0.01 seconds [Download](#)

7. Ejecutar una sentencia que ponga “alias” en los campos.

```

SELECT
    ID_CLIENTE AS "ID del Cliente",
    Nombre_y_Apellido AS "Nombre y Apellido",
    Razon_Social AS "Razón Social",
    e_mail AS "Correo Electrónico",
    Tarjeta_de_credito AS "Tarjeta de Crédito"
FROM Cliente;

```

```

1 SELECT
2     ID_CLIENTE AS "ID del Cliente",
3     Nombre_y_Apellido AS "Nombre y Apellido",
4     Razon_Social AS "Razón Social",
5     e_mail AS "Correo Electrónico",
6     Tarjeta_de_credito AS "Tarjeta de Crédito"
7 FROM Cliente;
8
9

```

ID del Cliente	Nombre y Apellido	Razón Social	Correo Electrónico	Tarjeta de Crédito
1	MARINAO GECIK	15-4-567-8976	MGECKIK@GMAIL.COM	3456-5678-3245-7654
2	RICARDO PEREZ	15-4-356-8900	RPEREZ@GMAIL.COM	1234-5678-0789-2345
4	PABLO PATRIZI	15-4-894-1334	PPATRIZI@GMAIL.COM	9986-5077-3896-1652
3	SUSANA VAINSHTEIN	15-4-765-1234	SWAINSHTEIN@GMAIL.COM	9876-5092-3456-1342

4 rows returned in 0.01 seconds [Download](#)

8. Ejecutar una sentencia que concatene el valor de 2 campos de cualquier tabla.

```

SELECT NOMBRE_Y_APELLIDO || ' - ' || E_MAIL FROM Cliente;

```

```
1 SELECT NOMBRE_Y_APELLIDO || ' - ' || E_MAIL FROM Cliente;
2
3
4
5
6
7
8
9
```

Results Explain Describe Saved SQL History

NOMBRE_Y_APELLIDO ' - ' E_MAIL
MARINAO GECIK - MGECEK@GMAIL.COM
RICARDO PEREZ - RPerez@GMAIL.COM
PABLO PATRIZI - PPATRIZI@GMAIL.COM
SUSANA VAINSHTEIN - SVAINSHTEIN@GMAIL.COM

4 rows returned in 0.01 seconds [Download](#)

9. Que cláusula usarías para eliminar los duplicados de valores que haya en un campo.

```
SELECT DISTINCT FECHA_PEDIDO FROM PEDIDO;
```

```
1 SELECT DISTINCT FECHA_PEDIDO FROM PEDIDO;
```

Results Explain Describe Saved SQL History

FECHA_PEDIDO
5/21/2018
6/21/2018
3/21/2019

3 rows returned in 0.00 seconds [Download](#)

10. Ejecuta una sentencia que filtre información por un campo que sea numérico.

```
SELECT PRECIO_TOTAL FROM PEDIDO WHERE PRECIO_TOTAL > 120;
```

1 `SELECT PRECIO_TOTAL FROM PEDIDO WHERE PRECIO_TOTAL > 120;`
2

Results Explain Describe Saved SQL History

PRECIO_TOTAL
200
300
250
150

4 rows returned in 0.00 seconds [Download](#)

11. Ejecuta una sentencia que filtre información por un campo que sea carácter.

```
SELECT * FROM CLIENTE WHERE NOMBRE_Y_APELLIDO LIKE '%A%';
```

1 `SELECT * FROM CLIENTE WHERE NOMBRE_Y_APELLIDO LIKE '%A%';`
2
3
4

Results Explain Describe Saved SQL History

ID_CLIENTE	NOMBRE_Y_APELLIDO	RAZON_SOCIAL	
1	MARINAO GECIK	15-4-567-8976	MGECIK
2	RICARDO PEREZ	15-4-356-8900	RPEREZ
4	PABLO PATRIZI	15-4-894-1334	PPATRIZ
3	SUSANA VAINSHTEIN	15-4-765-1234	SVAINSH

4 rows returned in 0.00 seconds [Download](#)

12. Ejecuta una sentencia que utilice el operador BETWEEN.

```
SELECT PRECIO_TOTAL FROM PEDIDO WHERE PRECIO_TOTAL BETWEEN 200 AND 300;
```

```

1 SELECT PRECIO_TOTAL FROM PEDIDO WHERE PRECIO_TOTAL BETWEEN 200 AND 300;
2
3
4

```

PRECIO_TOTAL
200
300
250

3 rows returned in 0.01 seconds [Download](#)

13. Ejecuta una sentencia que utilice el operador IN y NOT IN.

```

SELECT * FROM CATEGORIA_PROVEEDOR WHERE CODIGO_PROV IN('AC')
AND CODIGO_PROV NOT IN('BC');

```

```

1 SELECT * FROM CATEGORIA_PROVEEDOR WHERE CODIGO_PROV IN('AC')
2 AND CODIGO_PROV NOT IN('BC');
3
4
5

```

CODIGO_PROV	DESCRIPCION	CANT_DE_VINO_VENDIDO
AC	ALTA CALIDAD	100

1 rows returned in 0.01 seconds [Download](#)

14. Ejecuta una sentencia que utilice el operador LIKE que utilice el carácter '%'.
 SELECT * FROM BODEGA WHERE DESCRIPCION LIKE '%A';

The screenshot shows a SQL query editor with the following query:

```
1 SELECT * FROM BODEGA WHERE DESCRIPCION LIKE '%A';
2
```

Below the query editor, the results are displayed in a table with the following structure:

TIPO_B	
BA	BODEGA AMALAYA

1 rows returned in 0.02 seconds [Download](#)

15. Ejecuta una sentencia que utilice el operador IS NULL / IS NOT NULL.

```
SELECT * FROM CATEGORIA_PROVEEDOR WHERE FACTURACION_ANUAL IS NOT NULL;
```

The screenshot shows a SQL query editor with the following query:

```
1 SELECT * FROM CATEGORIA_PROVEEDOR WHERE FACTURACION_ANUAL IS NOT NULL;
2
3
```

Below the query editor, the results are displayed in a table with the following structure:

CODIGO_PROV	DESCRIPCION	CANT_DE_VINO_VENDIDO	FACTURACION_ANUAL
AC	ALTA CALIDAD	100	1000000
BA	BAJA CALIDAD	200	0

2 rows returned in 0.00 seconds [Download](#)

16. Ejecuta una sentencia que utilice la cláusula ORDER BY, ascendente, y otra descendente.

```
SELECT CANT_DE_VINO_VENDIDO
FROM CATEGORIA_PROVEEDOR
ORDER BY CANT_DE_VINO_VENDIDO ASC;
```

```

1 SELECT CANT_DE_VINO_VENDIDO FROM CATEGORIA_PROVEEDOR ORDER BY CANT_DE_VINO_VENDIDO ASC;
2
3

```

CANT_DE_VINO_VENDIDO
100
200

2 rows returned in 0.00 seconds [Download](#)

```

SELECT CANT_DE_VINO_VENDIDO
FROM CATEGORIA_PROVEEDOR
ORDER BY CANT_DE_VINO_VENDIDO DESC;

```

```

1 SELECT CANT_DE_VINO_VENDIDO FROM CATEGORIA_PROVEEDOR ORDER BY CANT_DE_VINO_VENDIDO DESC;
2
3

```

CANT_DE_VINO_VENDIDO
200
100

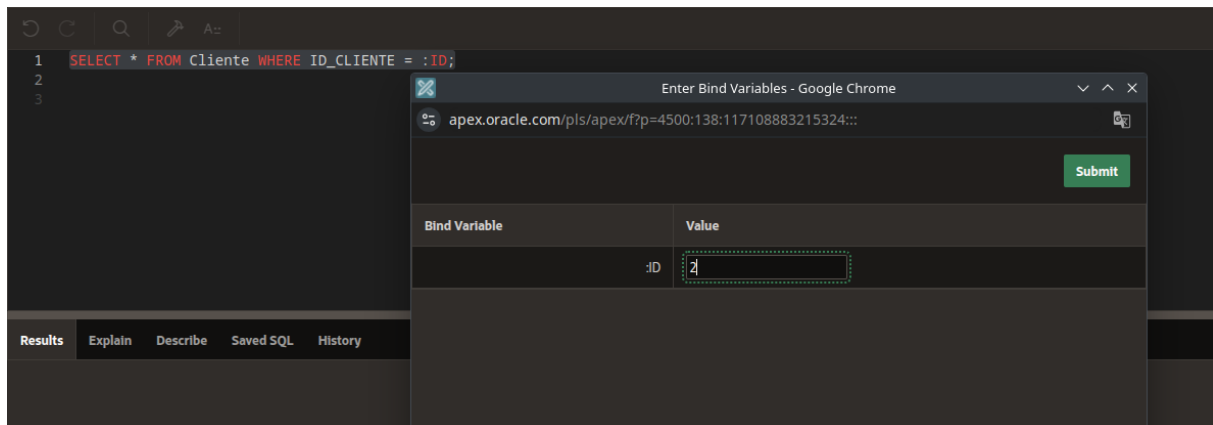
2 rows returned in 0.01 seconds [Download](#)

17. Ejecutar una sentencia que solicite un valor por pantalla.

```

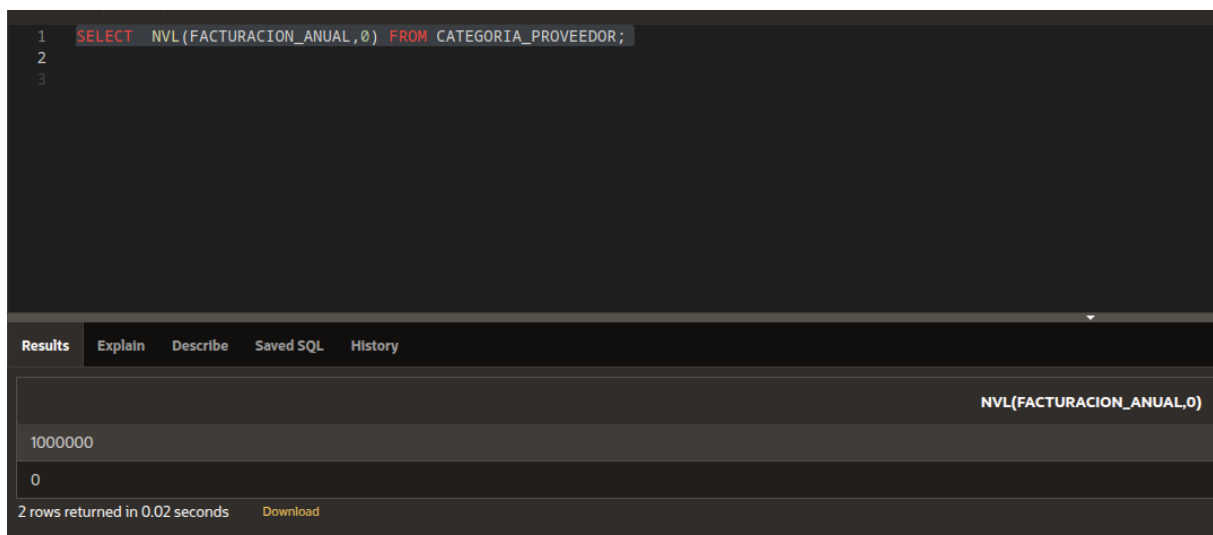
SELECT * FROM Cliente WHERE ID_CLIENTE = :ID;

```



18. Ejecutar una sentencia que utiliza la función NVL.

```
SELECT NVL(FACTURACION_ANUAL,0) FROM CATEGORIA_PROVEEDOR;
```



19. Ejecutar una sentencia que utiliza la función NVL2.

```
SELECT NVL2(FACTURACION_ANUAL,150,100) FROM CATEGORIA_PROVEEDOR;
```

```

1 SELECT NVL2(FACTURACION_ANUAL,150,100) FROM CATEGORIA_PROVEEDOR;
2
3

```

NVL2(FACTURACION_ANUAL,150,100)
150
150

2 rows returned in 0.01 seconds [Download](#)

20. Ejecutar una sentencia que utiliza la función NULLIF.

```

SELECT CANTIDAD,
       PRECIO_TOTAL / NULLIF(CANTIDAD, 0) AS PRECIO_POR_UNIDAD
FROM PEDIDO;

```

```

1 SELECT CANTIDAD,
2        PRECIO_TOTAL / NULLIF(CANTIDAD, 0) AS PRECIO_POR_UNIDAD
3 FROM PEDIDO;
4

```

CANTIDAD	PRECIO_POR_UNIDAD
100	2
120	2.5
130	1.92307692307692307692307692307692
100	1.5
120	1
140	.714285714285714285714285714285714

21. Ejecutar una sentencia que utiliza la función COALESCE.

```

SELECT COALESCE(CANTIDAD,NULL) FROM PEDIDO;

```

```
5 SELECT COALESCE(CANTIDAD, NULL) FROM PEDIDO;
```

COALESCE(CANTIDAD, NULL)
100
120
130
100
120
140

6 rows returned in 0.00 seconds [Download](#)

22. Ejecutar una sentencia que utiliza la función CASE.

```
SELECT CASE
  WHEN (CANTIDAD*PRECIO_TOTAL) > 1000 THEN 'SO GOOD'
  WHEN (CANTIDAD*PRECIO_TOTAL) < 1000 THEN 'GOOD'
  ELSE 'BAD'
END AS RESULTADO
FROM PEDIDO;
```

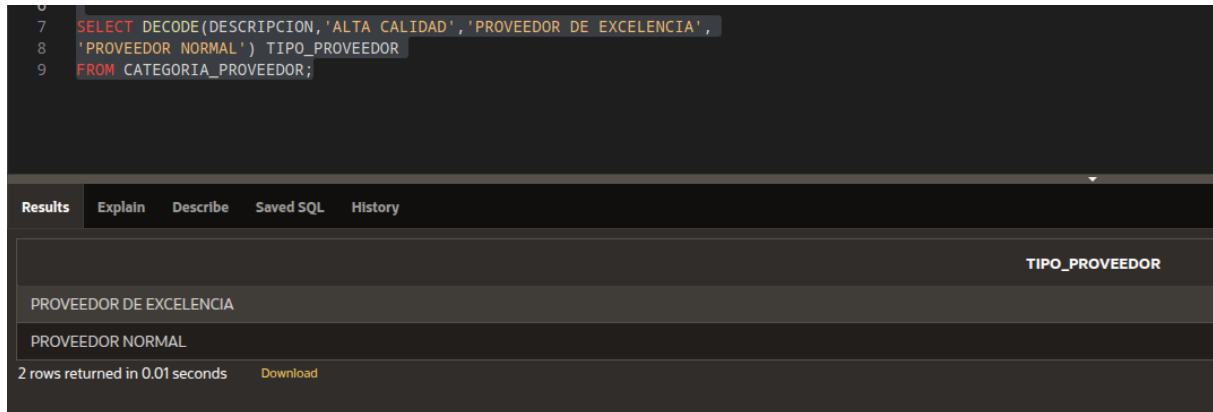
```
1 SELECT CANTIDAD,
2        PRECIO_TOTAL / NULLIF(CANTIDAD, 0) AS PRECIO_POR_UNIDAD
3 FROM PEDIDO;
4
5 SELECT CASE
6   WHEN (CANTIDAD*PRECIO_TOTAL) > 1000 THEN 'SO GOOD'
7   WHEN (CANTIDAD*PRECIO_TOTAL) < 1000 THEN 'GOOD'
8   ELSE 'BAD'
9 END AS RESULTADO
10 FROM PEDIDO;
```

RESULTADO
SO GOOD
SO GOOD
SO GOOD
SO GOOD
SO GOOD
SO GOOD

6 rows returned in 0.00 seconds [Download](#)

23. Ejecutar una sentencia que utiliza la función DECODE.

```
SELECT DECODE(DESCRIPCION,'ALTA CALIDAD','PROVEEDOR DE EXCELENCIA',
'PROVEEDOR NORMAL') TIPO_PROVEEDOR
FROM CATEGORIA_PROVEEDOR;
```

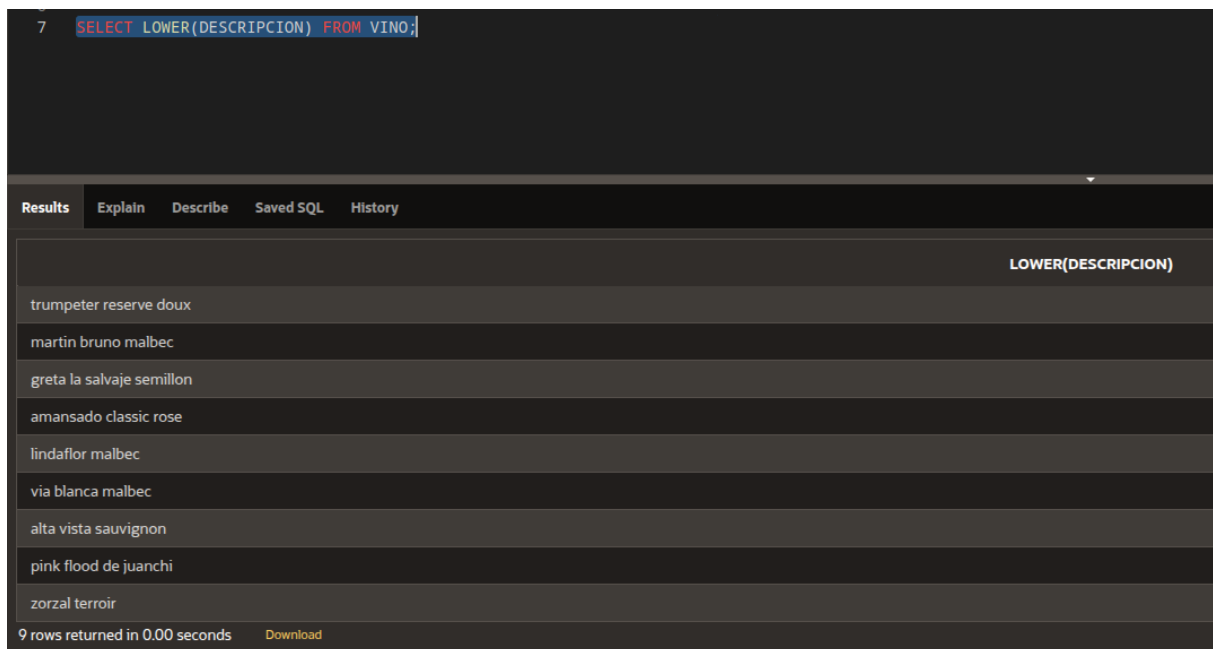


The screenshot shows a SQL query execution interface. The query is: `SELECT DECODE(DESCRIPCION,'ALTA CALIDAD','PROVEEDOR DE EXCELENCIA','PROVEEDOR NORMAL') TIPO_PROVEEDOR FROM CATEGORIA_PROVEEDOR;`. The results are displayed in a table with one column, **TIPO_PROVEEDOR**, containing two rows: `PROVEEDOR DE EXCELENCIA` and `PROVEEDOR NORMAL`. The interface also shows tabs for Results, Explain, Describe, Saved SQL, and History, and a status bar indicating 2 rows returned in 0.01 seconds.

TIPO_PROVEEDOR
PROVEEDOR DE EXCELENCIA
PROVEEDOR NORMAL

24. Ejecutar una sentencia que utiliza la función de Fila LOWER.

```
SELECT LOWER(DESCRIPCION) FROM VINO;
```



The screenshot shows a SQL query execution interface. The query is: `SELECT LOWER(DESCRIPCION) FROM VINO;`. The results are displayed in a table with one column, **LOWER(DESCRIPCION)**, containing nine rows of wine descriptions in lowercase: `trumpeter reserve doux`, `martin bruno malbec`, `greta la salvaje semillon`, `amansado classic rose`, `lindaflor malbec`, `via blanca malbec`, `alta vista sauvignon`, `pink flood de juanchi`, and `zorzal terroir`. The interface also shows tabs for Results, Explain, Describe, Saved SQL, and History, and a status bar indicating 9 rows returned in 0.00 seconds.

LOWER(DESCRIPCION)
trumpeter reserve doux
martin bruno malbec
greta la salvaje semillon
amansado classic rose
lindaflor malbec
via blanca malbec
alta vista sauvignon
pink flood de juanchi
zorzal terroir

25. Ejecutar una sentencia que utiliza la función de Fila UPPER.

```
SELECT UPPER(DESCRIPCION) FROM VINO;
```

1 `SELECT UPPER(DESCRIPCION) FROM VINO;`

UPPER(DESCRIPCION)
TRUMPETER RESERVE DOUX
MARTIN BRUNO MALBEC
GRETA LA SALVAJE SEMILLON
AMANSADO CLASSIC ROSE
LINDAFLORES MALBEC
VIA BLANCA MALBEC
ALTA VISTA SAUVIGNON
PINK FLOOD DE JUANCHI
ZORZAL TERROIR

26. Ejecutar una sentencia que utiliza la función de Fila INITCAP.

```
SELECT INITCAP(DESCRIPCION) FROM VINO;
```

1 `SELECT INITCAP(DESCRIPCION) FROM VINO;`

INITCAP(DESCRIPCION)
Trumpeter Reserve Doux
Martin Bruno Malbec
Greta La Salvaje Semillon
Amansado Classic Rose
Lindaflor Malbec
Via Blanca Malbec
Alta Vista Sauvignon
Pink Flood De Juanchi
Zorzal Terroir

9 rows returned in 0.02 seconds [Download](#)

27. Ejecutar una sentencia que utiliza la función de Fila CONCAT.

```
SELECT CONCAT(CODIGO,DESCRIPCION) FROM VINO;
```

```

1 SELECT CONCAT(CODIGO,DESCRIPCION) FROM VINO;
2
3

```

CONCAT(CODIGO,DESCRIPCION)
TRTRUMPETER RESERVE DOUX
MMMARTIN BRUNO MALBEC
GLGRETA LA SALVAJE SEMILLON
ASAMANSADO CLASSIC ROSE
LMLINDAFLOL MALBEC
VBVIA BLANCA MALBEC
AVALTA VISTA SAUVIGNON
PPPINK FLOOD DE JUANCHI
ZTZORZAL TERROIR

28. Ejecutar una sentencia que utiliza la función de Fila SUBSTR.

```
SELECT SUBSTR(DESCRIPCION,1,5) FROM VINO;
```

```

1 SELECT SUBSTR(DESCRIPCION,1,5) FROM VINO;
2
3

```

SUBSTR(DESCRIPCION,1,5)
TRUMP
MARTI
GRETA
AMANS
LINDA
VIA B
ALTA
PINK
ZORZA

29. Ejecutar una sentencia que utiliza la función de Fila LENGTH.

```
SELECT LENGTH(DESCRIPCION) FROM VINO;
```

```
1 SELECT LENGTH(DESCRIPCION) FROM VINO;
2 |
3
```

	LENGTH(DESCRIPCION)
22	
19	
25	
21	
16	
17	
20	
21	
14	

30. Ejecutar una sentencia que utiliza la función de Fila INSTR.

```
SELECT INSTR(DESCRIPCION,'TI') FROM VINO;
```

```
1 SELECT INSTR(DESCRIPCION,'TI') FROM VINO;
2
3
```

Results

Explain

Describe

Saved SQL

History

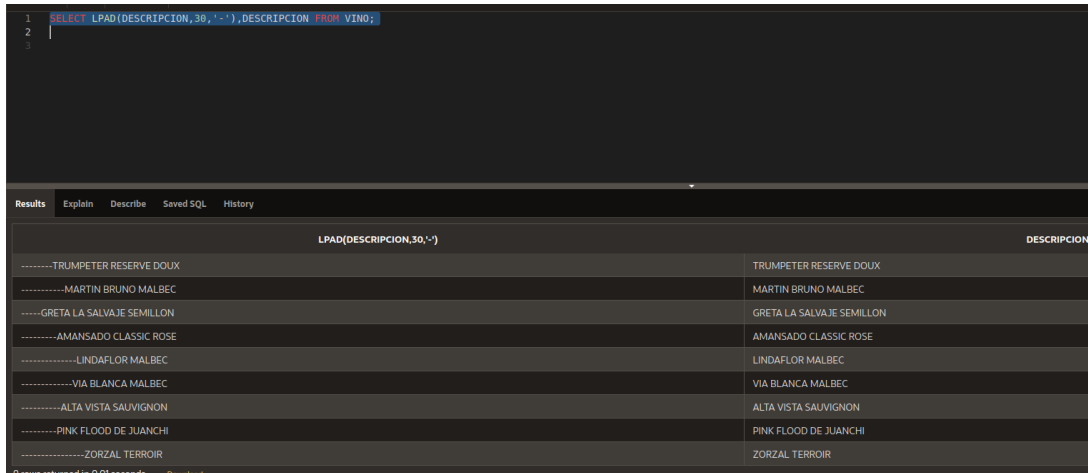
	INSTR(DESCRIPCION,'TI')
0	
4	
0	
0	
0	
0	
0	
0	
0	
0	

9 rows returned in 0.01 seconds

Download

31. Ejecutar una sentencia que utiliza la función de Fila LPAD.

```
SELECT LPAD(DESCRIPCION,30,'-'),DESCRIPCION FROM VINO;
```

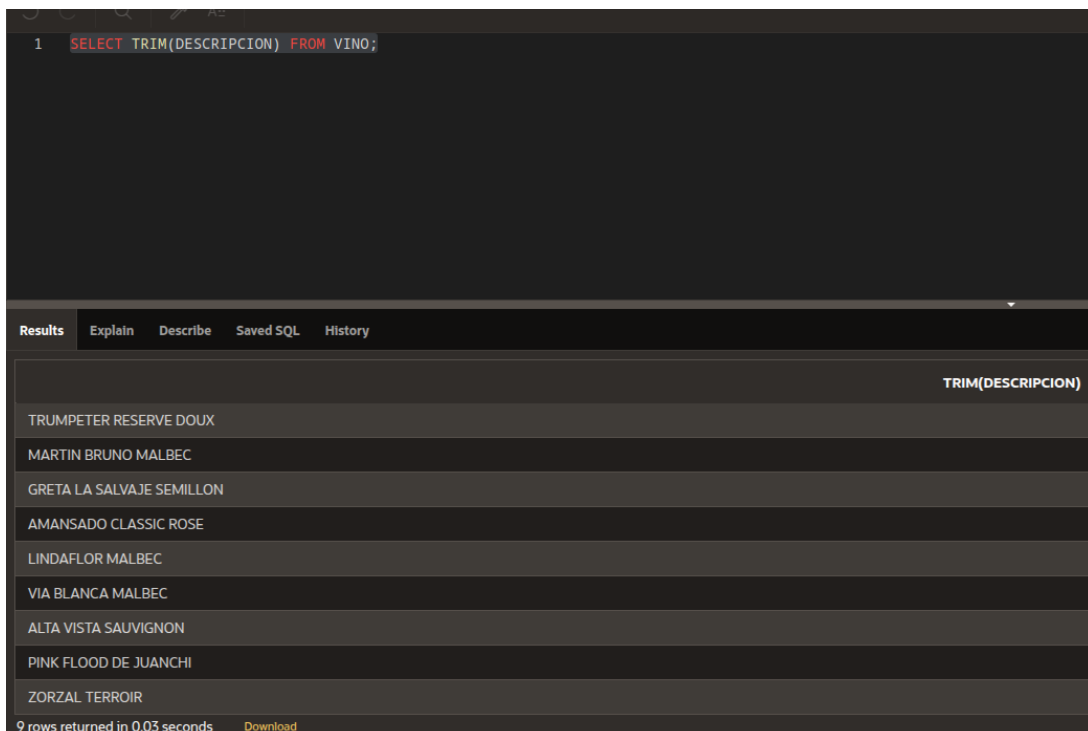


LPAD(DESCRIPCION,30,'-')	DESCRIPCION
-----TRUMPETER RESERVE DOUX	TRUMPETER RESERVE DOUX
-----MARTIN BRUNO MALBEC	MARTIN BRUNO MALBEC
-----GRETA LA SALVAJE SEMILLON	GRETA LA SALVAJE SEMILLON
-----AMANSADO CLASSIC ROSE	AMANSADO CLASSIC ROSE
-----LINDAFLOL MALBEC	LINDAFLOL MALBEC
-----VIA BLANCA MALBEC	VIA BLANCA MALBEC
-----ALTA VISTA SAUVIGNON	ALTA VISTA SAUVIGNON
-----PINK FLOOD DE JUANCHI	PINK FLOOD DE JUANCHI
-----ZORZAL TERROIR	ZORZAL TERROIR

9 rows returned in 0.01 seconds

32. Ejecutar una sentencia que utiliza la función de Fila TRIM, LTRIM y RTRIM.

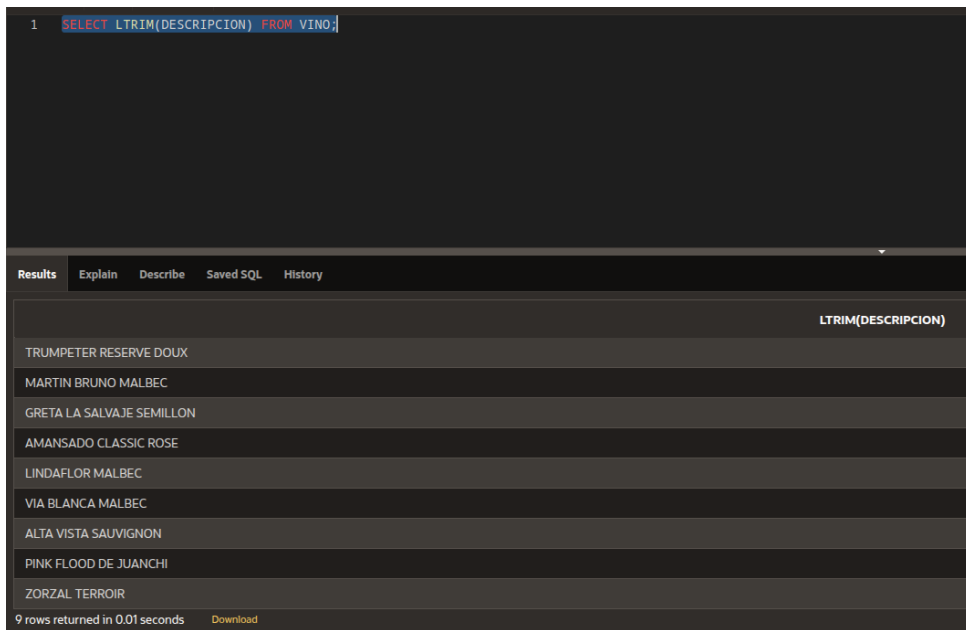
```
SELECT TRIM(DESCRIPCION) FROM VINO;
```



TRIM(DESCRIPCION)
TRUMPETER RESERVE DOUX
MARTIN BRUNO MALBEC
GRETA LA SALVAJE SEMILLON
AMANSADO CLASSIC ROSE
LINDAFLOL MALBEC
VIA BLANCA MALBEC
ALTA VISTA SAUVIGNON
PINK FLOOD DE JUANCHI
ZORZAL TERROIR

9 rows returned in 0.03 seconds

```
SELECT LTRIM(DESCRIPCION) FROM VINO;
```

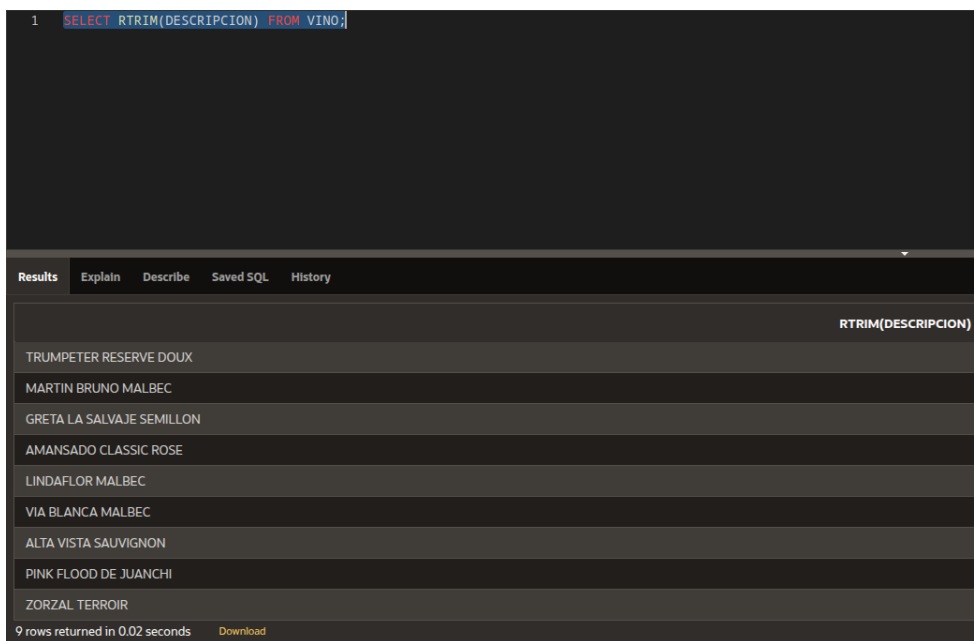


The screenshot shows a SQL query execution interface. The query entered is `SELECT LTRIM(DESCRIPCION) FROM VINO;`. The results are displayed in a table with a single column header `LTRIM(DESCRIPCION)`. The table contains 9 rows of wine descriptions. At the bottom, it states "9 rows returned in 0.01 seconds" and includes a "Download" link.

LTRIM(DESCRIPCION)
TRUMPETER RESERVE DOUX
MARTIN BRUNO MALBEC
GRETA LA SALVAJE SEMILLON
AMANSADO CLASSIC ROSE
LINDAFLOR MALBEC
VIA BLANCA MALBEC
ALTA VISTA SAUVIGNON
PINK FLOOD DE JUANCHI
ZORZAL TERROIR

9 rows returned in 0.01 seconds [Download](#)

```
SELECT RTRIM(DESCRIPCION) FROM VINO;
```



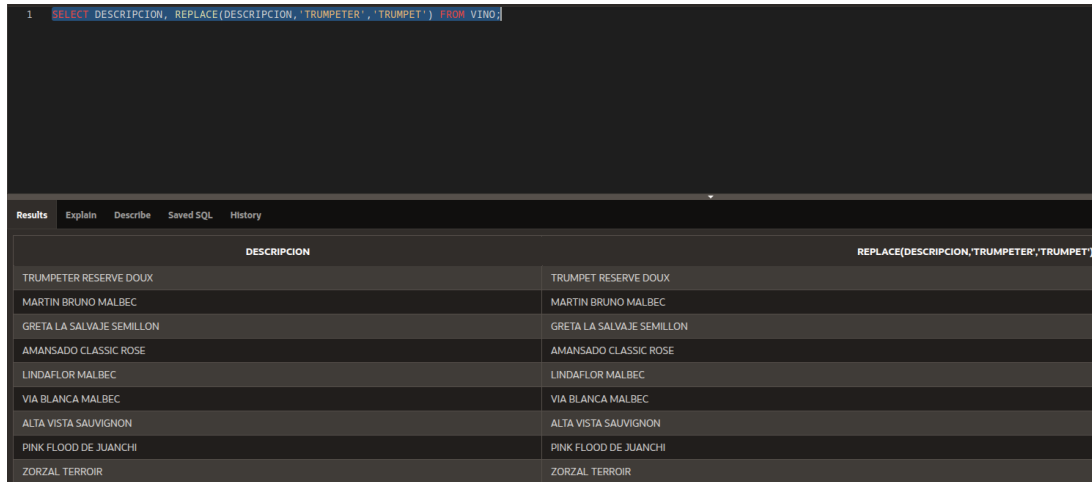
The screenshot shows a SQL query execution interface. The query entered is `SELECT RTRIM(DESCRIPCION) FROM VINO;`. The results are displayed in a table with a single column header `RTRIM(DESCRIPCION)`. The table contains 9 rows of wine descriptions. At the bottom, it states "9 rows returned in 0.02 seconds" and includes a "Download" link.

RTRIM(DESCRIPCION)
TRUMPETER RESERVE DOUX
MARTIN BRUNO MALBEC
GRETA LA SALVAJE SEMILLON
AMANSADO CLASSIC ROSE
LINDAFLOR MALBEC
VIA BLANCA MALBEC
ALTA VISTA SAUVIGNON
PINK FLOOD DE JUANCHI
ZORZAL TERROIR

9 rows returned in 0.02 seconds [Download](#)

33. Ejecutar una sentencia que utiliza la función de Fila REPLACE.

```
SELECT DESCRIPCION, REPLACE(DESCRIPCION,'TRUMPETER','TRUMPET') FROM VINO;
```

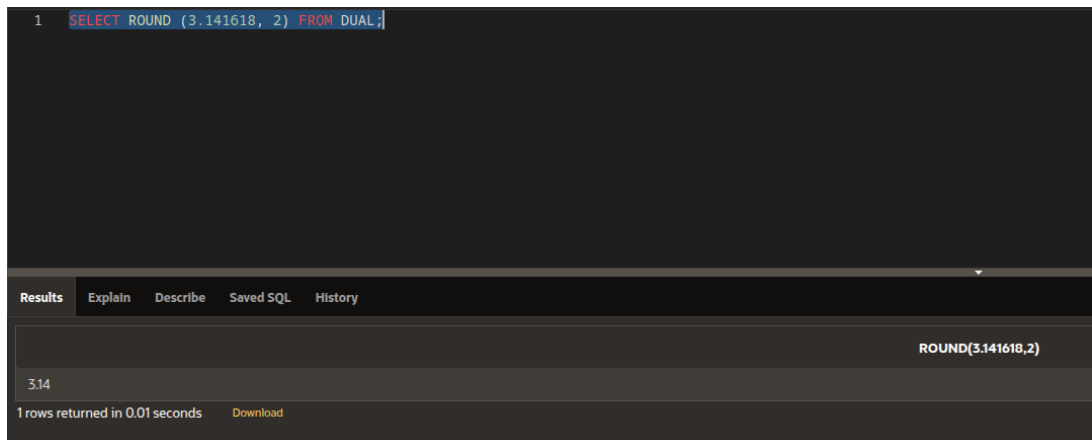


The screenshot shows a SQL query execution interface. The query is: `1 SELECT DESCRIPCION, REPLACE(DESCRIPCION,'TRUMPETER','TRUMPET') FROM VINO;`. The results are displayed in a table with two columns: `DESCRIPCION` and `REPLACE(DESCRIPCION,'TRUMPETER','TRUMPET')`. The table contains 10 rows of data, where the first row shows the replacement of 'TRUMPETER' with 'TRUMPET' in the description 'TRUMPETER RESERVE DOUX'.

DESCRIPCION	REPLACE(DESCRIPCION,'TRUMPETER','TRUMPET')
TRUMPETER RESERVE DOUX	TRUMPET RESERVE DOUX
MARTIN BRUNO MALBEC	MARTIN BRUNO MALBEC
GRETA LA SALVAJE SEMILLON	GRETA LA SALVAJE SEMILLON
AMANSADO CLASSIC ROSE	AMANSADO CLASSIC ROSE
LINDAFLORES MALBEC	LINDAFLORES MALBEC
VIA BLANCA MALBEC	VIA BLANCA MALBEC
ALTA VISTA SAUVIGNON	ALTA VISTA SAUVIGNON
PINK FLOOD DE JUANCHI	PINK FLOOD DE JUANCHI
ZORZAL TERROIR	ZORZAL TERROIR

34. Ejecutar una sentencia que utiliza la función de Fila ROUND.

```
SELECT ROUND (3.141618, 2) FROM DUAL;
```



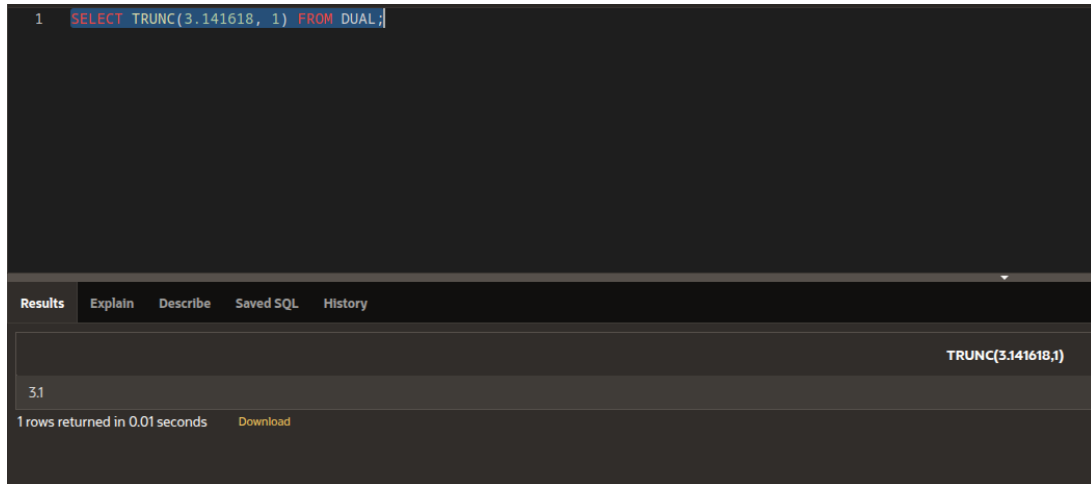
The screenshot shows a SQL query execution interface. The query is: `1 SELECT ROUND (3.141618, 2) FROM DUAL;`. The results are displayed in a table with one column: `ROUND(3.141618,2)`. The table contains one row with the value 3.14. Below the table, it says "1 rows returned in 0.01 seconds" and there is a "Download" link.

ROUND(3.141618,2)
3.14

1 rows returned in 0.01 seconds [Download](#)

35. Ejecutar una sentencia que utiliza la función de Fila TRUNC.

```
SELECT TRUNC(3.141618, 1) FROM DUAL;
```



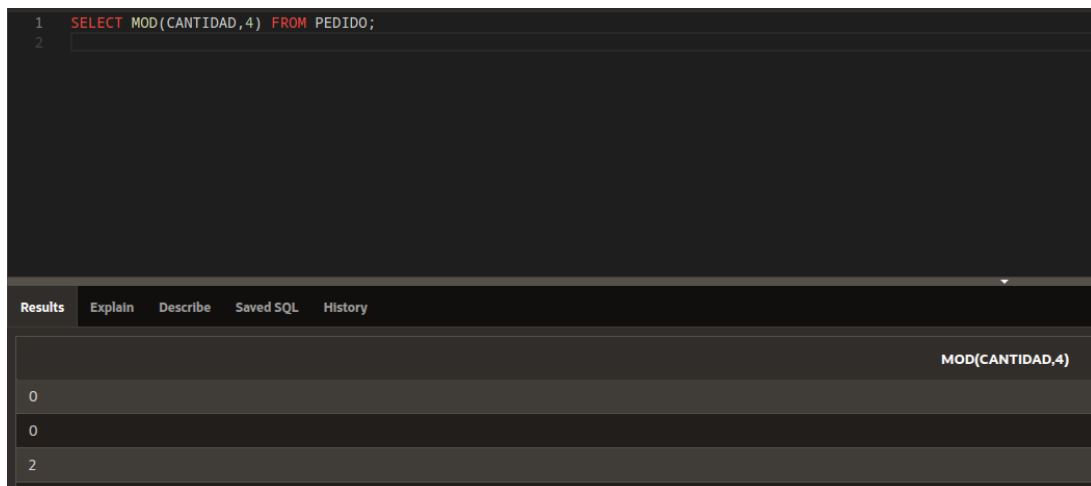
The screenshot shows a SQL query execution interface. The query is: `1 SELECT TRUNC(3.141618, 1) FROM DUAL;`. The interface has tabs for Results, Explain, Describe, Saved SQL, and History. The Results tab is active, showing a table with one column named `TRUNC(3.141618,1)` and one row with the value `3.1`. Below the table, it says "1 rows returned in 0.01 seconds" and there is a "Download" link.

TRUNC(3.141618,1)
3.1

1 rows returned in 0.01 seconds [Download](#)

36. Ejecutar una sentencia que utiliza la función de Fila MOD.

```
SELECT MOD(CANTIDAD,4) FROM PEDIDO;
```

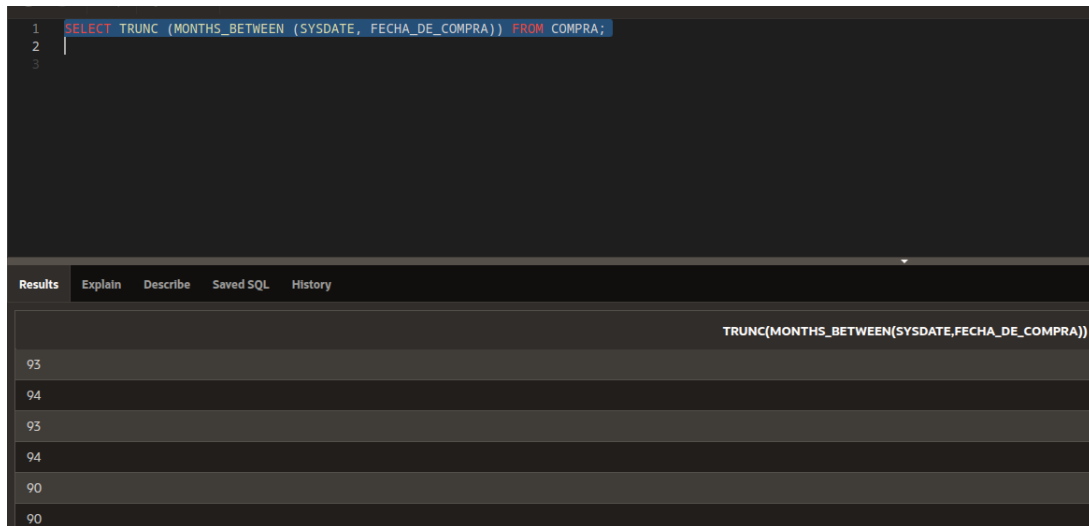


The screenshot shows a SQL query execution interface. The query is: `1 SELECT MOD(CANTIDAD,4) FROM PEDIDO;`. The interface has tabs for Results, Explain, Describe, Saved SQL, and History. The Results tab is active, showing a table with one column named `MOD(CANTIDAD,4)` and three rows with the values `0`, `0`, and `2`.

MOD(CANTIDAD,4)
0
0
2

37. Ejecutar una sentencia que utiliza la función de Fecha MONTHS_BETWEEN.

```
SELECT TRUNC (MONTHS_BETWEEN (SYSDATE, FECHA_DE_COMPRA)) FROM COMPRA;
```

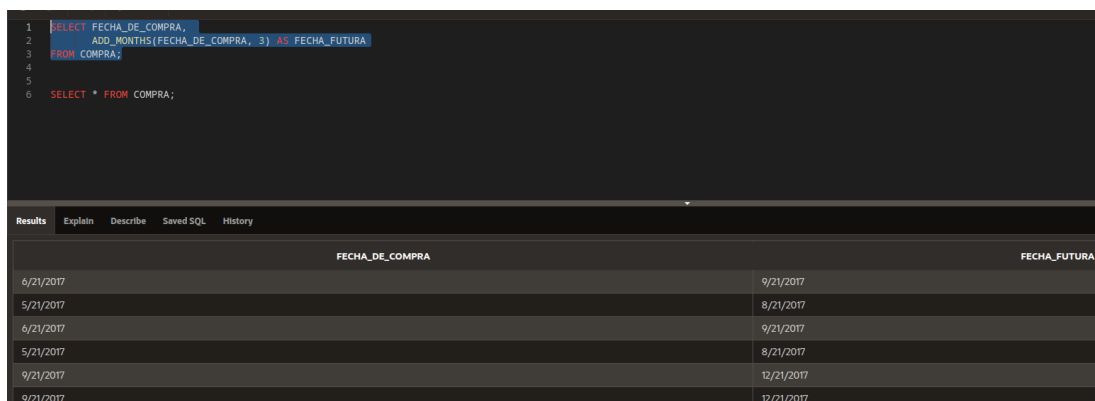


The screenshot shows a SQL query execution interface. The query is: `SELECT TRUNC (MONTHS_BETWEEN (SYSDATE, FECHA_DE_COMPRA)) FROM COMPRA;`. The results are displayed in a table with one column: `TRUNC(MONTHS_BETWEEN(SYSDATE,FECHA_DE_COMPRA))`. The results are: 93, 94, 93, 94, 90, 90.

TRUNC(MONTHS_BETWEEN(SYSDATE,FECHA_DE_COMPRA))
93
94
93
94
90
90

38. Ejecutar una sentencia que utiliza la función de Fecha ADD_MONTHS

```
SELECT FECHA_DE_COMPRA,  
       ADD_MONTHS(FECHA_DE_COMPRA, 3) AS FECHA_FUTURA  
FROM COMPRA;
```

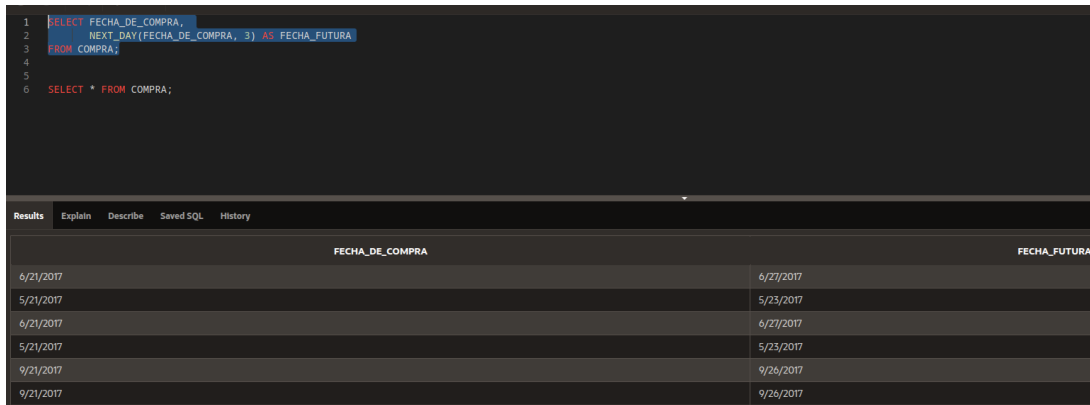


The screenshot shows a SQL query execution interface. The query is: `SELECT FECHA_DE_COMPRA, ADD_MONTHS(FECHA_DE_COMPRA, 3) AS FECHA_FUTURA FROM COMPRA;`. The results are displayed in a table with two columns: `FECHA_DE_COMPRA` and `FECHA_FUTURA`. The results are: 6/21/2017, 9/21/2017; 5/21/2017, 8/21/2017; 6/21/2017, 9/21/2017; 5/21/2017, 8/21/2017; 9/21/2017, 12/21/2017; 9/21/2017, 12/21/2017.

FECHA_DE_COMPRA	FECHA_FUTURA
6/21/2017	9/21/2017
5/21/2017	8/21/2017
6/21/2017	9/21/2017
5/21/2017	8/21/2017
9/21/2017	12/21/2017
9/21/2017	12/21/2017

39. Ejecutar una sentencia que utiliza la función de Fecha NEXT_DAY y LAST_DAY

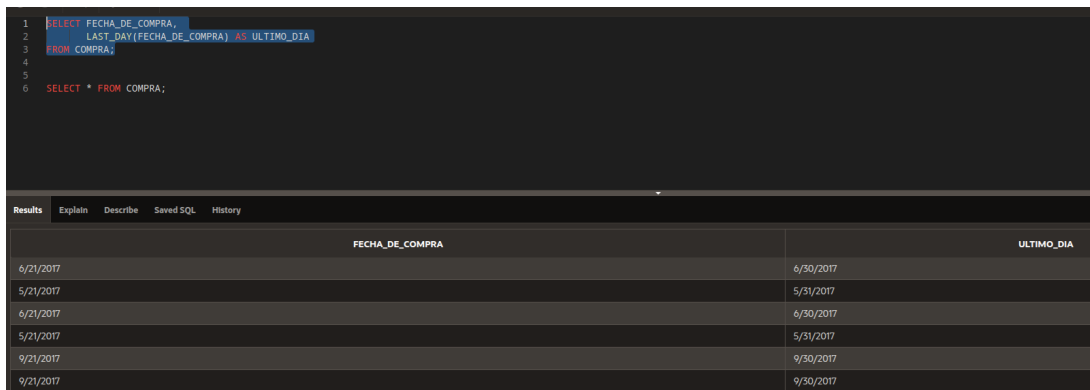
```
SELECT FECHA_DE_COMPRA,  
       NEXT_DAY(FECHA_DE_COMPRA, 3) AS FECHA_FUTURA  
FROM COMPRA;
```



The screenshot shows a SQL query execution interface. The query is: `SELECT FECHA_DE_COMPRA, NEXT_DAY(FECHA_DE_COMPRA, 3) AS FECHA_FUTURA FROM COMPRA;`. The results are displayed in a table with two columns: `FECHA_DE_COMPRA` and `FECHA_FUTURA`. The results are as follows:

FECHA_DE_COMPRA	FECHA_FUTURA
6/21/2017	6/27/2017
5/21/2017	5/23/2017
6/21/2017	6/27/2017
5/21/2017	5/23/2017
9/21/2017	9/26/2017
9/21/2017	9/26/2017

```
SELECT FECHA_DE_COMPRA,  
       LAST_DAY(FECHA_DE_COMPRA) AS ULTIMO_DIA  
FROM COMPRA;
```

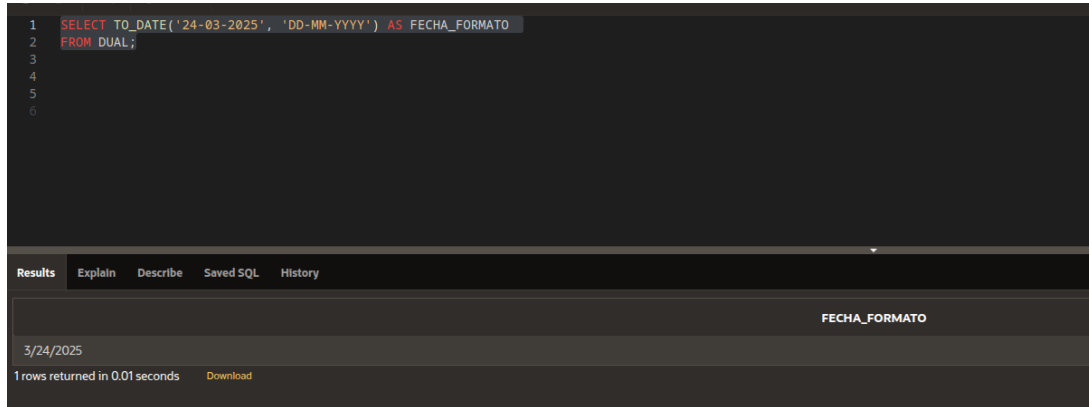


The screenshot shows a SQL query execution interface. The query is: `SELECT FECHA_DE_COMPRA, LAST_DAY(FECHA_DE_COMPRA) AS ULTIMO_DIA FROM COMPRA;`. The results are displayed in a table with two columns: `FECHA_DE_COMPRA` and `ULTIMO_DIA`. The results are as follows:

FECHA_DE_COMPRA	ULTIMO_DIA
6/21/2017	6/30/2017
5/21/2017	5/31/2017
6/21/2017	6/30/2017
5/21/2017	5/31/2017
9/21/2017	9/30/2017
9/21/2017	9/30/2017

40. Ejecutar una sentencia que utiliza la función TO_DATE.

```
SELECT TO_DATE('24-03-2025', 'DD-MM-YYYY') AS FECHA_FORMATO
FROM DUAL;
```

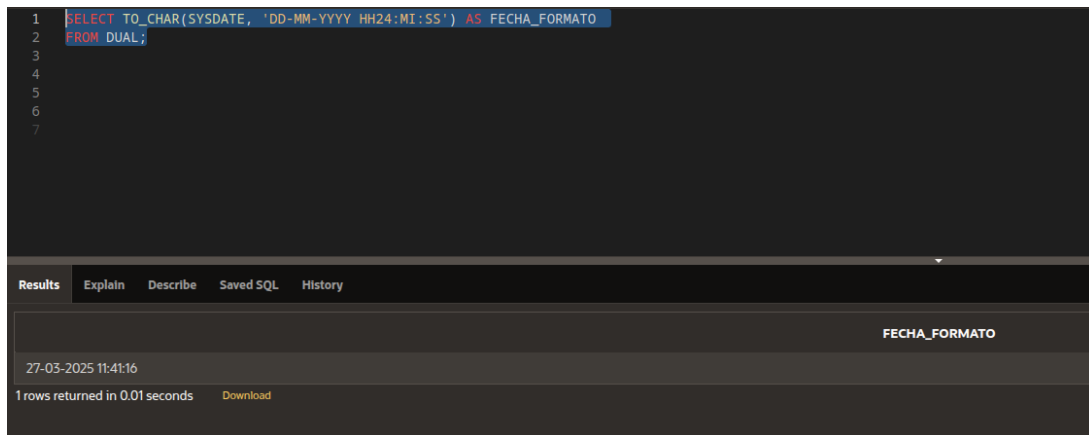


The screenshot shows a SQL query execution interface. The query is: `SELECT TO_DATE('24-03-2025', 'DD-MM-YYYY') AS FECHA_FORMATO FROM DUAL;`. The results section shows a single row with the value `3/24/2025` under the column header `FECHA_FORMATO`. The interface includes tabs for Results, Explain, Describe, Saved SQL, and History. The status bar indicates "1 rows returned in 0.01 seconds" and a "Download" button.

FECHA_FORMATO
3/24/2025

41. Ejecutar una sentencia que utiliza la función TO_CHAR.

```
SELECT TO_CHAR(SYSDATE, 'DD-MM-YYYY HH24:MI:SS') AS FECHA_FORMATO
FROM DUAL;
```

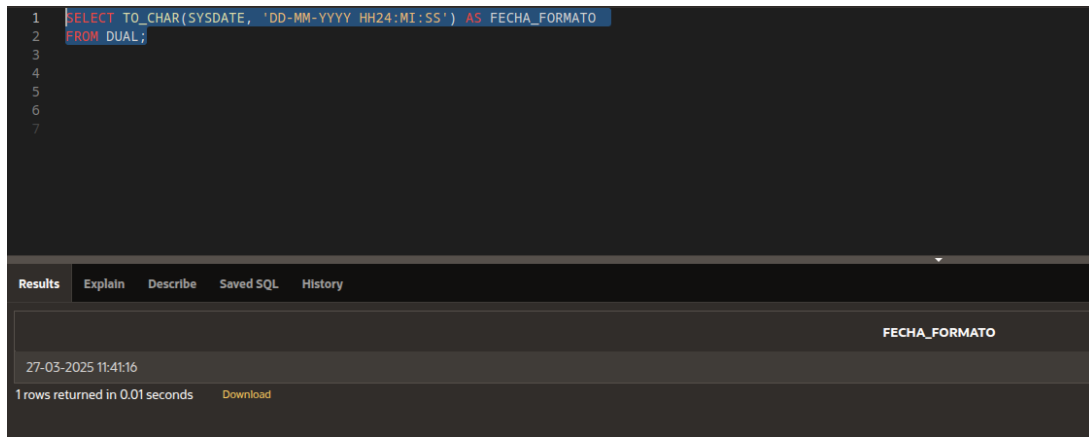


The screenshot shows a SQL query execution interface. The query is: `SELECT TO_CHAR(SYSDATE, 'DD-MM-YYYY HH24:MI:SS') AS FECHA_FORMATO FROM DUAL;`. The results section shows a single row with the value `27-03-2025 11:41:16` under the column header `FECHA_FORMATO`. The interface includes tabs for Results, Explain, Describe, Saved SQL, and History. The status bar indicates "1 rows returned in 0.01 seconds" and a "Download" button.

FECHA_FORMATO
27-03-2025 11:41:16

42. Ejecutar una sentencia que utiliza la función TO_NUMBER

```
SELECT TO_NUMBER(REPLACE(TARJETA_DE_CREDITO, '-', '')) AS NUMERO_DE_TARJ  
FROM CLIENTE;
```

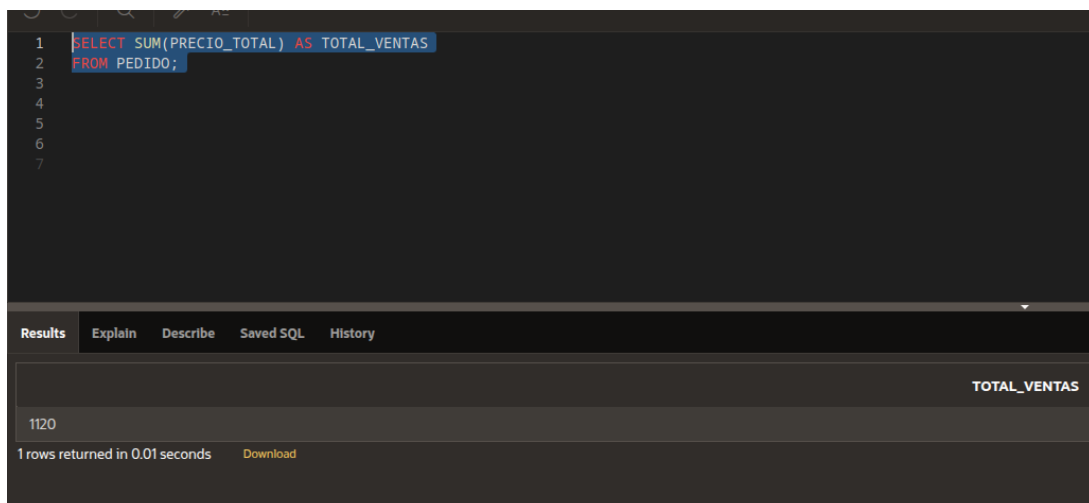


The screenshot shows a SQL query execution interface. The query is: `SELECT TO_CHAR(SYSDATE, 'DD-MM-YYYY HH24:MI:SS') AS FECHA_FORMATO FROM DUAL;`. The results table has one column, `FECHA_FORMATO`, and one row with the value `27-03-2025 11:41:16`. The interface also shows "1 rows returned in 0.01 seconds" and a "Download" button.

FECHA_FORMATO
27-03-2025 11:41:16

43. Ejecutar una sentencia que utilice la función de grupo SUM.

```
SELECT SUM(PRECIO_TOTAL) AS TOTAL_VENTAS  
FROM PEDIDO;
```



The screenshot shows a SQL query execution interface. The query is: `SELECT SUM(PRECIO_TOTAL) AS TOTAL_VENTAS FROM PEDIDO;`. The results table has one column, `TOTAL_VENTAS`, and one row with the value `1120`. The interface also shows "1 rows returned in 0.01 seconds" and a "Download" button.

TOTAL_VENTAS
1120

44. Ejecutar una sentencia que utiliza la función de grupo AVG.

```
SELECT AVG(PRECIO_TOTAL) AS AVG_VENTAS
FROM PEDIDO;
```

```
1 SELECT AVG(PRECIO_TOTAL) AS AVG_VENTAS
2 FROM PEDIDO;
3
4
5
6
7
```

AVG_VENTAS
186.666666666666666666666666666667

1 rows returned in 0.02 seconds [Download](#)

45. Ejecutar una sentencia que utiliza la función de grupo MIN y MAX.

```
SELECT MIN(PRECIO_TOTAL) AS AVG_VENTAS
FROM PEDIDO;
```

```
SELECT MAX(PRECIO_TOTAL) AS AVG_VENTAS
FROM PEDIDO;
```

```
1 SELECT MIN(PRECIO_TOTAL) AS AVG_VENTAS
2 FROM PEDIDO;
3
4 SELECT MAX(PRECIO_TOTAL) AS AVG_VENTAS
5 FROM PEDIDO;
6
7
8
```

Results

Explain

Describe

Saved SQL

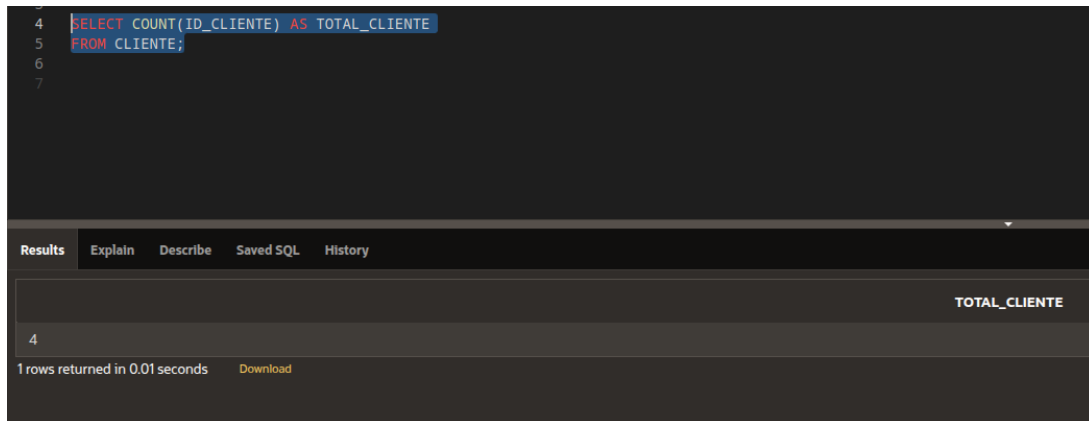
History

AVG_VENTAS
300

1 rows returned in 0.03 seconds
[Download](#)

46. Ejecutar una sentencia que utiliza la función de grupo COUNT.

```
SELECT COUNT(ID_CLIENTE) AS TOTAL_CLIENTE  
FROM CLIENTE;
```



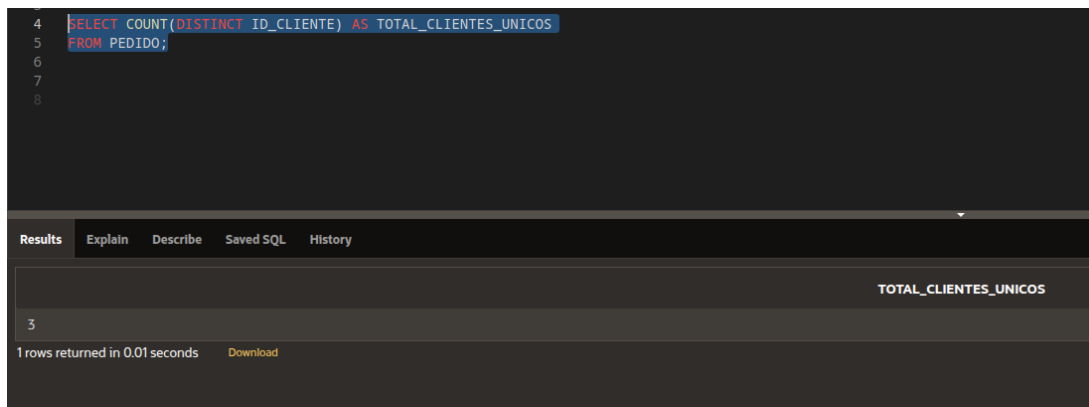
The screenshot shows a SQL query execution interface. The query is: `SELECT COUNT(ID_CLIENTE) AS TOTAL_CLIENTE FROM CLIENTE;`. The results are displayed in a table with one column, `TOTAL_CLIENTE`, and one row with the value `4`. The interface also shows tabs for Results, Explain, Describe, Saved SQL, and History. At the bottom, it indicates "1 rows returned in 0.01 seconds" and a "Download" button.

TOTAL_CLIENTE
4

1 rows returned in 0.01 seconds [Download](#)

47. Ejecutar una sentencia que utiliza la función de grupo COUNT combinado con DISTINCT.

```
SELECT COUNT(DISTINCT ID_CLIENTE) AS TOTAL_CLIENTES_UNICOS  
FROM PEDIDO;
```



The screenshot shows a SQL query execution interface. The query is: `SELECT COUNT(DISTINCT ID_CLIENTE) AS TOTAL_CLIENTES_UNICOS FROM PEDIDO;`. The results are displayed in a table with one column, `TOTAL_CLIENTES_UNICOS`, and one row with the value `3`. The interface also shows tabs for Results, Explain, Describe, Saved SQL, and History. At the bottom, it indicates "1 rows returned in 0.01 seconds" and a "Download" button.

TOTAL_CLIENTES_UNICOS
3

1 rows returned in 0.01 seconds [Download](#)

48. Ejecutar una sentencia que utiliza la cláusula HAVING.

```
SELECT ID_CLIENTE, SUM (PRECIO_TOTAL) AS TOTAL_VENDIDO
FROM PEDIDO
GROUP BY ID_CLIENTE
HAVING SUM (PRECIO_TOTAL) > 150
```



The screenshot shows a SQL query editor with the following code:

```
4 SELECT ID_CLIENTE, SUM (PRECIO_TOTAL) AS TOTAL_VENDIDO
5 FROM PEDIDO
6 GROUP BY ID_CLIENTE
7 HAVING SUM (PRECIO_TOTAL) > 150
8
9
10 select * from pedido;
```

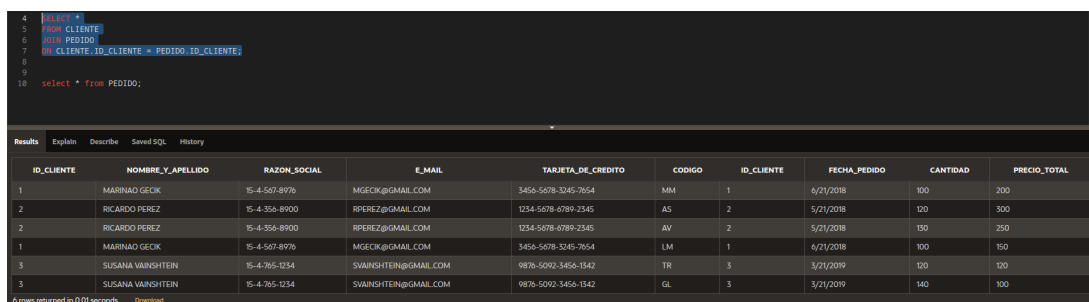
Below the editor, the 'Results' tab is active, displaying the following table:

ID_CLIENTE	TOTAL_VENDIDO
1	350
2	550
3	220

3 rows returned in 0.01 seconds [Download](#)

49. Ejecutar una sentencia que haga un JOIN.

```
SELECT *
FROM CLIENTE
JOIN PEDIDO
ON CLIENTE.ID_CLIENTE = PEDIDO.ID_CLIENTE;
```



The screenshot shows a SQL query editor with the following code:

```
4 SELECT *
5 FROM CLIENTE
6 JOIN PEDIDO
7 ON CLIENTE.ID_CLIENTE = PEDIDO.ID_CLIENTE
8
9
10 select * from PEDIDO;
```

Below the editor, the 'Results' tab is active, displaying the following table:

ID_CLIENTE	NOMBRE_Y_APELLIDO	RAZON_SOCIAL	E_MAIL	TARJETA_DE_CREDITO	CODIGO	ID_CLIENTE	FECHA_PEDIDO	CANTIDAD	PRECIO_TOTAL
1	MARINAO GECIK	15-4-567-8976	MGECK@GMAIL.COM	3456-5678-3245-7654	MM	1	4/21/2018	100	200
2	RICARDO PEREZ	15-4-356-8900	RPEREZ@GMAIL.COM	1234-5678-6789-2345	AS	2	5/21/2018	120	300
2	RICARDO PEREZ	15-4-356-8900	RPEREZ@GMAIL.COM	1234-5678-6789-2345	AV	2	5/21/2018	130	250
1	MARINAO GECIK	15-4-567-8976	MGECK@GMAIL.COM	3456-5678-3245-7654	LM	1	6/21/2018	100	150
3	SUSANA VAINSHTEIN	15-4-765-1234	SVAINSHTEIN@GMAIL.COM	9876-5002-3456-1342	TR	3	3/21/2019	120	120
3	SUSANA VAINSHTEIN	15-4-765-1234	SVAINSHTEIN@GMAIL.COM	9876-5002-3456-1342	GL	3	3/21/2019	140	100

6 rows returned in 0.01 seconds [Download](#)

50. Ejecutar una sentencia que haga un LEFT OUTER JOIN.

```
SELECT *
FROM CLIENTE
LEFT OUTER JOIN PEDIDO
ON CLIENTE.ID_CLIENTE = PEDIDO.ID_CLIENTE;
```

```
4 SELECT *
5 FROM CLIENTE
6 LEFT OUTER JOIN PEDIDO
7 ON CLIENTE.ID_CLIENTE = PEDIDO.ID_CLIENTE;
8
9
10 select * from PEDIDO;
```

ID_CLIENTE	NOMBRE_Y_APELLIDO	RAZON_SOCIAL	E_MAIL	TARJETA_DE_CREDITO	CODIGO	ID_CLIENTE	FECHA_PEDIDO	CANTIDAD	PRECIO_TOTAL
1	MARINAO GECK	TS-4-567-8976	MGECK@GMAIL.COM	3456-5678-3245-7654	MM	1	6/21/2018	100	200
2	RICARDO PEREZ	TS-4-356-8900	RPerez@GMAIL.COM	1234-5678-0789-2345	AS	2	5/21/2018	120	300
2	RICARDO PEREZ	TS-4-356-8900	RPerez@GMAIL.COM	1234-5678-0789-2345	AV	2	5/21/2018	130	250
1	MARINAO GECK	TS-4-567-8976	MGECK@GMAIL.COM	3456-5678-3245-7654	LM	1	6/21/2018	100	150
3	SUSANA VAINSHTEIN	TS-4-765-1234	SVAINSHTEIN@GMAIL.COM	9876-5092-3456-1542	TR	3	3/21/2019	120	120
3	SUSANA VAINSHTEIN	TS-4-765-1234	SVAINSHTEIN@GMAIL.COM	9876-5092-3456-1542	GL	3	3/21/2019	140	100
4	PABLO PATRIZI	TS-4-894-1534	PPATRIZI@GMAIL.COM	9986-5077-3896-1652	-	-	-	-	-

7 rows returned in 0.07 seconds [Download](#)

51. Ejecutar una sentencia que haga un RIGHT OUTER JOIN.

```
SELECT *
FROM CLIENTE
RIGHT OUTER JOIN PEDIDO
ON CLIENTE.ID_CLIENTE = PEDIDO.ID_CLIENTE;
```

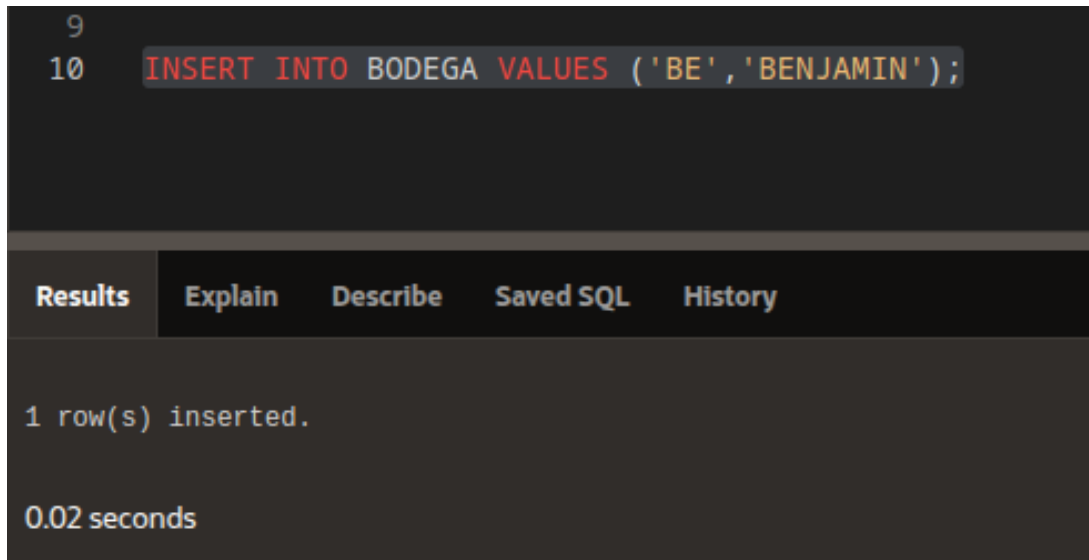
```
4 SELECT *
5 FROM CLIENTE
6 RIGHT OUTER JOIN PEDIDO
7 ON CLIENTE.ID_CLIENTE = PEDIDO.ID_CLIENTE;
8
9
10 select * from PEDIDO;
```

ID_CLIENTE	NOMBRE_Y_APELLIDO	RAZON_SOCIAL	E_MAIL	TARJETA_DE_CREDITO	CODIGO	ID_CLIENTE	FECHA_PEDIDO	CANTIDAD	PRECIO_TOTAL
1	MARINAO GECK	TS-4-567-8976	MGECK@GMAIL.COM	3456-5678-3245-7654	MM	1	6/21/2018	100	200
1	MARINAO GECK	TS-4-567-8976	MGECK@GMAIL.COM	3456-5678-3245-7654	LM	1	6/21/2018	100	150
2	RICARDO PEREZ	TS-4-356-8900	RPerez@GMAIL.COM	1234-5678-0789-2345	AS	2	5/21/2018	120	300
2	RICARDO PEREZ	TS-4-356-8900	RPerez@GMAIL.COM	1234-5678-0789-2345	AV	2	5/21/2018	130	250
3	SUSANA VAINSHTEIN	TS-4-765-1234	SVAINSHTEIN@GMAIL.COM	9876-5092-3456-1542	TR	3	3/21/2019	120	120
3	SUSANA VAINSHTEIN	TS-4-765-1234	SVAINSHTEIN@GMAIL.COM	9876-5092-3456-1542	GL	3	3/21/2019	140	100

7 rows returned in 0.07 seconds [Download](#)

52. Ejecutar una sentencia de INSERT.

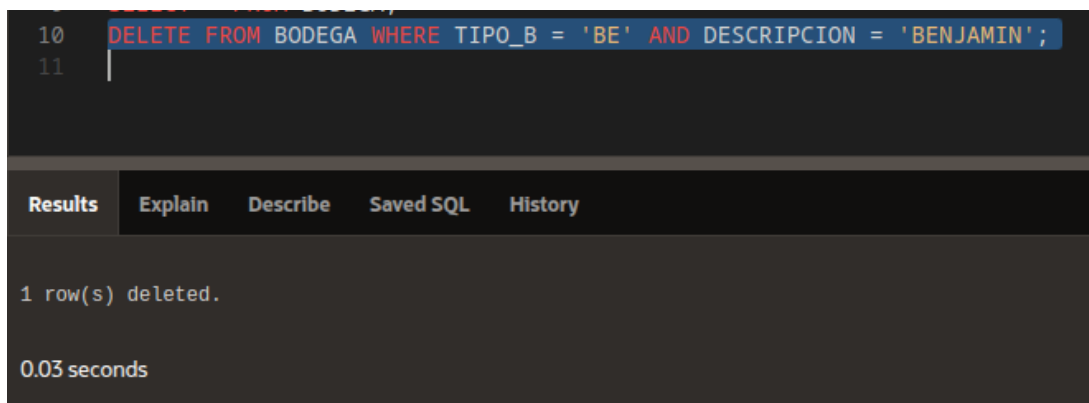
```
INSERT INTO BODEGA VALUES ('BE','BENJAMIN');
```



The screenshot shows a SQL IDE interface. At the top, a SQL statement is entered: `INSERT INTO BODEGA VALUES ('BE','BENJAMIN');`. Below the editor, there is a tabbed interface with 'Results' selected. The results pane displays the message '1 row(s) inserted.' and the execution time '0.02 seconds'.

53. Ejecutar una sentencia de DELETE.

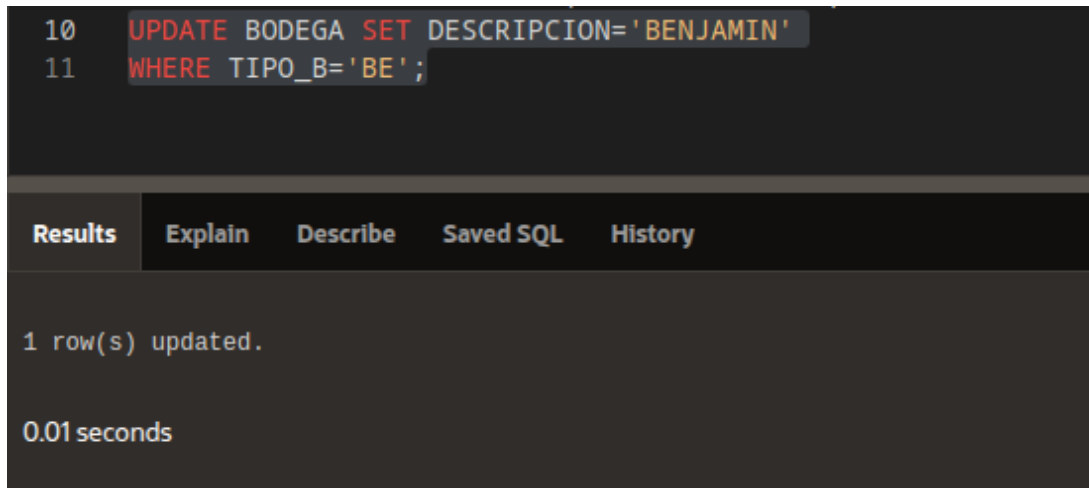
```
DELETE FROM BODEGA WHERE TIPO_B = 'BE' AND DESCRIPCION = 'BENJAMIN';
```



The screenshot shows a SQL IDE interface. At the top, a SQL statement is entered: `DELETE FROM BODEGA WHERE TIPO_B = 'BE' AND DESCRIPCION = 'BENJAMIN';`. Below the editor, there is a tabbed interface with 'Results' selected. The results pane displays the message '1 row(s) deleted.' and the execution time '0.03 seconds'.

54. Ejecutar una sentencia de UPDATE.

```
UPDATE BODEGA SET DESCRIPCION='BENJAMIN'  
WHERE TIPO_B='BE';
```



The screenshot shows a SQL execution interface with a dark theme. At the top, two lines of SQL code are displayed with line numbers 10 and 11. Line 10 contains 'UPDATE BODEGA SET DESCRIPCION='BENJAMIN'' and line 11 contains 'WHERE TIPO_B='BE';'. Below the code, there is a horizontal tab bar with five tabs: 'Results', 'Explain', 'Describe', 'Saved SQL', and 'History'. The 'Results' tab is currently selected. Below the tabs, the execution result is shown as '1 row(s) updated.' followed by the execution time '0.01 seconds'.

```
10 UPDATE BODEGA SET DESCRIPCION='BENJAMIN'  
11 WHERE TIPO_B='BE';
```

Results	Explain	Describe	Saved SQL	History
1 row(s) updated.				
0.01 seconds				